THE AI TOOLBOX · CLASS 08

Data Cleaning & Preprocessing

Turn raw, messy data into something a model can **actually learn from**.

Merit Point Academy · 60 min · press **S** notes · → next · 💬 to comment

RECAP · WHERE WE LEFT OFF

Last class in 60 seconds

Two axes of data

Structured vs. unstructured; quantitative vs. qualitative — **the type picks the tools**.

The judgment layer

License · collection story · baked-in bias. Unclear license = pick another dataset.

Your datasheet

Source, variables, honest limitations — your project's data has papers now.

Homework share: what bias did you find in your dataset's collection story? And did `df.head()` match the documentation's promises — or did you catch your first surprise? **Today we fix the surprises.**

AGENDA · HOW THIS HOUR RUNS

Where we're going

- **0–8'** Homework share + the model that read the wrong thing
- **8–16'** Garbage in, garbage out · the five enemies of clean data
- **16–32'** The three moves: missing · normalize · **encode**
- **32–40'** Outliers & duplicates · feature engineering
- **40–46'** Demos: profiling a messy file · AI co-pilot cleaning
- **46–54'** Two labs: clean the AV log · *prove* scaling matters
- **54–60'** Your before/after report, homework & wrap-up
- Quizzes & 💬 comments throughout

By the end: handle missing values sensibly, put numbers on a comparable scale, encode categories without inventing fake order — and justify every change in a before/after report.

WARM-UP · ACTIVATE

Quick check · tap an answer

Your detection log has blank distances, a box with negative width, and 'pedestrian' spelled three different ways. You train a model on it as-is. What happens?

- The model fixes the data automatically — that's what learning means
- Training crashes immediately, so no harm done
- It trains fine and looks OK — but learns from garbage: three 'different' pedestrian classes, impossible boxes, silent skew
- Nothing; models only read the clean rows

THE STORY

The model that read the wrong thing

> A hospital trained an AI to spot serious pneumonia on chest X-rays, and it worked **suspiciously well**. When researchers looked closer, the model wasn't reading lungs at all — it had learned to spot a tiny **metal token** that portable X-ray machines stamp on the image. Sicker patients got portable scans, so the token predicted severity. **The data, not the disease, was doing the work.**

🔊 **Speak this story** Neural voice · click to play / pause

"Suspiciously well" is the tell — remember the overfitting quiz (Class 3)? A too-good number is a symptom, not a triumph. This failure mode has a name: **shortcut learning**. Your data is your model's entire world; whatever hides in it, the model will find.

CONCEPT 1 · KNOW YOUR ENEMIES

Garbage in, garbage out — the five enemies

🕳️ Missing values

Blank cells from sensor drops, skipped survey questions. (Old friends — Class 5.)

👯 Duplicates

The same row twice quietly **doubles its vote** in every statistic and every gradient step.

🚀 Outliers

A 900 km/h pedestrian: sensor glitch, typo — or a real, rare event. *Which one decides the fix.*

🔤 Inconsistent formats

'pedestrian' / 'Pedestrian ' / 'PED'; dates as 2026-07-26 and 7/26/26 — humans read through it, models can't.

⚖️ Logical errors

Negative box widths, ages of 250, confidence of −1: values that violate reality itself.

⏱️ The honest number

Researchers spend **~half their time here** (Class 3, pitfall #1). Cleaning isn't pre-research — **it IS research.**

CONCEPT 2 · THE SPINE OF PREPROCESSING

Three moves make data model-ready

1. **Handle missing**

   drop the row, or impute (fill) a value — *and disclose which*

2. **Normalize numbers**

   put columns on a comparable scale (0–1, or mean 0)

3. **Encode categories**

   "pedestrian" → numbers a model can eat — *without inventing order*

Why these three? Models are math: they can't average a blank, they're bullied by big-ranged columns, and they can't multiply the word "pedestrian." The three moves translate *reality's mess* into *math's requirements* — each gets its own slide now.

MOVE 1 OF 3 · MISSING VALUES, LEVEL 2

Drop or fill — now with judgment

🗑️ Drop when…

```
df = df.dropna(subset=["dist_m"])
```

Few rows affected, or the column is essential and unfakeable. Honest, simple — costs you data.

🩹 Fill (impute) when…

```
df["age"] = df["age"]
  .fillna(df["age"].median())
```

Many rows affected. **Median** beats mean when outliers lurk (one 900 km/h pedestrian drags the mean; the median shrugs).

Level-2 question — WHY is it missing? Random sensor drops are safe to fill. But if night frames are missing distance *because darkness breaks the ranging*, the holes carry information — filling them hides a pattern your night-detection study needs to know about. Ask why before you fix.

MOVE 2 OF 3 · NORMALIZATION

Put numbers on a comparable scale

Min-max → squeeze into 0–1

```
x_scaled = (x - x.min())
        / (x.max() - x.min())
```

Simple, bounded, great default for features with known ranges (pixel values, box coordinates).

Z-score → mean 0, spread 1

```
x_scaled = (x - x.mean()) / x.std()
```

Centers each column and equalizes spread; the robust choice when outliers or open-ended ranges exist.

Why it's not cosmetic: distance_m runs 0–100; confidence runs 0–1. Any model that compares or combines them hears distance **shouting 100× louder** — confidence becomes a whisper. Lab 2 today *proves* this: same data, same model, and scaling flips the answer.

MOVE 3 OF 3 · ENCODING CATEGORIES

"Pedestrian" → numbers, without lying

❌ The tempting trap

```
pedestrian → 1
car        → 2
sign       → 3
```

Looks harmless — but you just told the model **sign = 3 × pedestrian** and car sits "between" them. You invented an order that *doesn't exist* (nominal data, Class 7!).

✅ One-hot encoding

```
pd.get_dummies(df["object"])

is_ped  is_car  is_sign
   1       0       0
```

One column per category, a single 1 per row. No fake order, no fake distances — each class stands alone.

The connection: this is why Class 7's nominal-vs-ordinal distinction earns its keep — **ordinal** data (dusk / night / deep-night) may keep its order as 1-2-3; **nominal** data must never get one. The data type picks the encoding.

CHECK · THE ENCODING TRAP

Quick check · tap an answer

Your dataset has two category columns: object_class (pedestrian/car/sign) and lighting (dusk/night/deep-night). How should each be encoded?

- Both as 1, 2, 3 — numbers are numbers
- Both one-hot — categories are categories
- object_class one-hot (nominal, no real order); lighting may be 1-2-3 (ordinal — the order is real)
- Neither needs encoding; models read words now

CONCEPT 3 · THE REMAINING ENEMIES

Outliers and duplicates: detect, then decide

🚀 Outliers

```
df.describe()  # min/max scream first
df[df["conf"] > 1]  # impossible!
```

Impossible values (conf = −1, width < 0) → fix or drop, no mercy. **Possible-but-extreme** values → investigate: a glitch to remove, or a rare event your study is *about*? A 3 a.m. pedestrian is exactly who night-detection must not delete.

👯 Duplicates

```
df.duplicated().sum()  # how many?
df = df.drop_duplicates()
```

Exact copies double-count votes — from merged files, retried uploads, sensor stutters. Usually the easiest fix of the day… *after* you ask why they exist (a stuttering sensor is its own finding).

The shared rule: detection is mechanical; the **decision** is research. Every drop is a claim — "this value doesn't belong to the phenomenon I study" — and claims need reasons. That's why your deliverable is a report, not just a script.

CONCEPT 4 · CLEANING'S CREATIVE SIBLING

Feature engineering: build better columns

📅 Split what's bundled

timestamp → hour, weekday, **is_night**. One opaque column becomes three the model can use — and one is your study's key variable.

➗ Combine what's related

box_w × box_h → area; box_h / box_w → **aspect ratio** (your Lab-1 discovery from Class 7 — tall = pedestrian, wide = car).

🪣 Bucket what's continuous

distance_m → near / mid / far. Loses precision, gains robustness and readability — a trade you choose deliberately.

The professional's secret (Lesson 7's words): good feature engineering often improves results **more than a fancier model** — it's Class 3's pitfall #2 ("baseline first") meeting its positive twin: *better columns first*.

CHECK · NAME THE ENEMY

Quick check · tap an answer

df.describe() on your log shows: conf max = 1.0 ✓, dist_m min = −4.2, and value_counts shows 'car' 312 times and 'Car' 47 times. Diagnose the two problems.

- Two outliers
- A logical error (negative distance can't exist) and an inconsistent format ('car' vs 'Car' splitting one class)
- Missing values and duplicates
- The data is fine; models tolerate small issues

CASE STUDY · OUR RUNNING EXAMPLE, REFINED

Making the detection log model-ready

1 **Missing**

Blank distance_m readings: night-ranging issue? Investigate, then median-fill — *and disclose*.

2 **Normalize**

Box coordinates → 0–1 (min-max); distances z-scored so nothing shouts.

3 **Encode**

object_class one-hot: is_pedestrian / is_car / is_cyclist / is_sign.

4 **Report**

Rows in → rows out, every change with its reason. ↩ (This IS the deliverable.)

The thread continues: Class 5 read this log · Class 7 wrote its datasheet · today it becomes **trainable** — and in the model classes (10–13), this exact cleaned table is what learning runs on. Lab 1 performs all four steps, live.

DISCUSS · 5 MINUTES · THINK LIKE THE TOKEN

What shortcut could a pedestrian detector learn?

The X-ray model found a token instead of pneumonia. Your pedestrian detector is equally lazy — **what could it cheat with** instead of actually finding people?

Hunt the shortcut

Think about what *co-occurs* with pedestrians in dashcam data… crosswalk stripes? streetlights? city backgrounds? low speeds?

Design the catch

How would you expose it? What test image would fool a cheating model but not an honest one? What in the *data* would tip you off?

Pairs, 3 minutes — best shortcut and best catch in 💬.

LIVE DEMO 1

Profiling: interrogate before you fix

Five lines that expose all five enemies — run them on any new dataset before touching anything.

```
You ▸ df.info() → types + non-null counts (holes and wrong types jump out)
You ▸ df.isna().sum() → the hole map, column by column
You ▸ df.describe() → min/max scream: negative widths? conf > 1?
You ▸ df.duplicated().sum() → how many double votes?
You ▸ df["object"].value_counts() → 'car' vs 'Car' vs 'CAR ' in one glance
```

colab.research.google.com

LIVE DEMO 2

The co-pilot cleans — you approve

Class 4's programmer hat, pointed at cleaning. The AI writes pandas; you supply the judgment it doesn't have.

```
You ▸ Here's my df.info() and value_counts output: [paste]. List the data-quality problems you see.
```

```
You ▸ Write pandas to: lowercase+strip object labels, drop rows with negative box_w, median-fill dist_m — with a comment explaining each line.
```

```
You ▸ Why did you choose median over mean here? When would mean be better?
```

```
You ▸ What did you NOT fix that I should look at myself? (The judgment question.)
```

chatgpt.com

HANDS-ON LAB 1 · THE WHOLE REFINERY

Clean the messy detection log, end to end

The AV log as it **really** arrives: typos, holes, impossible values, duplicates. Profile it, fix it, and print the before/after report — your deliverable in miniature.

```python
import pandas as pd
import numpy as np

raw = pd.DataFrame({
    "object": ["pedestrian", "Pedestrian ", "car", "car", "PED",
               "sign", "car", "pedestrian"],
    "conf":   [0.94, 0.62, 0.88, 0.88, 0.58, 0.79, -1.0, 0.97],
    "dist_m": [8.2, np.nan, 15.0, 15.0, 7.8, np.nan, 12.0, 6.9],
    "box_w":  [116, 98, 100, 100, 90, 70, -40, 120],
})

print("=== BEFORE ===  rows:", len(raw))
```

▶ Run | 🎧 Explain each line | ✨ Explain | 🐞 Debug | ⚡ Improve | 🤖 Mira

Python loads on first Run

▶ Run the code to see the output here.

HANDS-ON LAB 2 · THE RECEIPTS

Prove normalization matters: same model, flipped answer

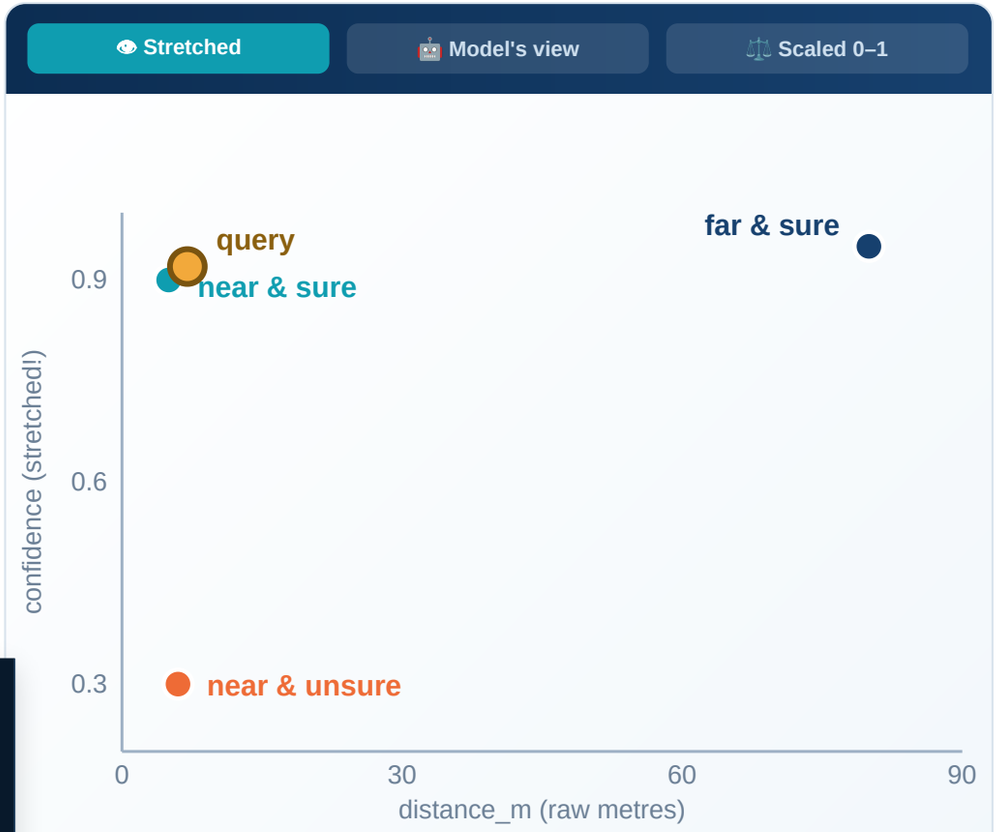
A tiny nearest-neighbor classifier (Class 13 preview!). Same data, same question — the **only** change is scaling. The chart shows the data **before and after**; press 🎧 and it follows the audio, line by line.

```python
import math

# Known detections: [distance_m (0-100), confidence (0-1)] -> situation
known = [
    ([80.0, 0.95], "far & sure"),
    ([ 5.0, 0.90], "near & sure"),
    ([ 6.0, 0.30], "near & unsure"),
]
query = [7.0, 0.92]   # near and confident - obviously "near & sure"...

def nearest(q, points):
```

▶ Run | 🎧 Explain each line | ✨ Explain | 🐞 Debug | ⚡ Improve | 🤖 Mira

Python loads on first Run

```
▶ Run the code to see the output here.
```



The data, the way humans plot it — axes stretched to fit. Three labeled detections + one query. Press 🎧 **Explain each line** for the guided tour.

HANDS-ON LAB 3 · THE RECEIPTS, ON REAL DATA

Two claims from this class, proved on 891 real passengers

Lab 2 proved scaling matters on three toy points. Now the same argument on the **real Kaggle Titanic set** — every number below is computed from the actual fares, not quoted at you. Two receipts: what one £512 ticket does to **min-max**, and why the "obvious" **C=0, Q=1, S=2** ordering is measurably false.

```
# Real Kaggle Titanic data — all 891 passengers. Scroll past these three
# lines; they are just the raw numbers, exactly as train.csv has them.
FARES = [float(x) for x in "7.25,71.28,7.92,53.1,8.05,8.46,51.86,21.07,11.13,30.07,16.7,26.55,8.05,31.27,7.85,16,29.12,13,18,7
PORTS = "SCSSSQSSSCSSSSSSQSSCSSQSSSCSQSCCQSCSCSSCSSCCQSQQCSSSCSCSSCSSC?SSCCSSSSSSSCSSSSSSSSQSSSSSSSSSSSSSCCSSSSSSSSSSQSCSSCSQ
LIVED = "0111000011110001010101110100100110001001000110010000110110100100011010000010001101101100100000000110000000110100000000

# == RECEIPT 1 == One outlier can eat your entire min-max range.
lo, hi = min(FARES), max(FARES)
order  = sorted(FARES)
median = order[len(order) // 2]
p75    = order[int(len(order) * 0.75)]
```

▶ Run | ✨ Explain | 🐞 Debug | ⚡ Improve | 🤖 Mira | Python loads on first Run

```
▶ Run the code to see the output here.
```

YOUR PROJECT

The before/after cleaning report

Clean **your own dataset** (the one from your Class-7 datasheet) — and show exactly what you changed and **why**.

- **Profile** it: the 5-line ritual → your "before" snapshot
- Handle **missing values** — drop or fill, with the reason
- **Normalize** numbers · **encode** categories (type-appropriately!)
- Record **before vs. after**: rows kept, values fixed, columns transformed

Deliverable

A before/after report: rows kept, values fixed, columns transformed — **each with its reason**.

Success looks like

Every cleaning choice justified, and the "after" data has no missing values or mixed types.

BUILD IT NOW · HANDS-ON

Refine your own oil

1. Open your Class-7 Colab (dataset already loaded) → run the **5-line profiling ritual**; screenshot = your "before"
2. Fix formats first (`str.lower/strip`), then duplicates — *order matters (Lab 1!)*
3. Impossible values: drop with a printed reason · Missing: drop or fill, **disclose**
4. Normalize numeric columns · one-hot the nominal ones (check your datasheet's types!)
5. Re-run the ritual = your "after" · write the report table: change → count → reason
6. Runtime → **Run all**, commit notebook + report to your GitHub repo

Colab | pandas | ChatGPT / Claude (co-pilot) | GitHub

Stuck on a weird value? That's not friction — that's a finding. The strangest thing in your data is often the most interesting sentence in your report.

BENCHMARK · JUDGE AGAINST THIS

What a finished cleaning report looks like

Dataset: *Night-Time Pedestrians v2* · 1,847 rows in → **1,791 rows out**

Formats: 3 class spellings → 2 clean classes (`str.lower`+map). *Reason: one concept, one label.*

Duplicates: 38 exact copies dropped. *Reason: re-uploaded batch double-counted frames (found in upload log).*

Impossible: 18 boxes with w≤0 dropped. *Reason: annotation-tool export bug; unfixable.*

Missing: 74 lighting tags filled "night". *Reason: all from one camera whose tag field broke; its footage is verifiably night-only. Disclosed.*

Normalized: box coords min-max to 0–1. *Reason: comparable across image sizes.*

Encoded: class one-hot (nominal); lighting kept ordinal 1–3. *Reason: datasheet types.*

The bar: counts for every change, a reason that cites **evidence** (upload logs, camera checks) not vibes — and one kept-outlier note: *"3 a.m. pedestrians retained: rare but real, and they're the study."*

REFERENCE · KEY TERMS

Today's vocabulary

Imputation — filling missing values (mean/median/mode) instead of dropping; always disclosed.

Normalization / standardization — min-max to 0–1 / z-score to mean-0, spread-1: making columns comparable.

One-hot encoding — one 0/1 column per category; numbers without fake order.

Outlier — a value far from the rest: impossible (drop), glitch (fix), or rare-but-real (keep — maybe it's the study).

Feature engineering — building better columns: splitting, combining, bucketing. Often beats a fancier model.

Shortcut learning — a model exploiting an accidental signal (the X-ray token) instead of the real one.

RECOMMENDED HOMEWORK

Before next class

1. **Finish your before/after report** (your dataset, every change justified) and submit the Colab link in Google Classroom; commit both to GitHub.
2. **One feature-engineering experiment:** build one new column (split / combine / bucket) and write one sentence on what question it could help answer.
3. **Shortcut audit:** one sentence — what accidental signal might hide in *your* dataset, and what test would expose it?

⭐ Stretch (optional)

Kaggle's **"Store Sales — Time Series Forecasting"**: open a high-voted notebook and read ONLY its cleaning section. Count the five enemies — how many appear, and what did they justify?

kaggle.com · Store Sales

Task 3 is the X-ray story turned on your own project — the audit habit that separates "it works" from "I know why it works." Questions → 💬 on any slide.

TODAY'S LINE TO REMEMBER

"Errors using inadequate data are much less than those using no data at all."

— Charles Babbage, father of the computer

Imperfect data, honestly cleaned and disclosed, beats both fantasy and paralysis. Refine what you have.

🔊 Speak this

WRAP-UP

Three things to keep

- Dirty data **fails silently** — profile first, fix second, disclose always
- Three moves: **missing · normalize · encode** — and the data type picks the encoding
- Every drop is a **claim with a reason** — and beware the model that works *suspiciously well*

Exit ticket: the strangest thing you found in your data today — and what you decided to do about it. Post in 💬.

Next · Class 9 — data visualization: make your clean data **tell its story**. One honest chart beats a table of digits.

BACKUP MATERIAL

Extra slides — if there's time

Deeper dives and extra practice. Skip in a tight hour; pull up on demand or for a longer session.

Auditing for shortcuts

how the token was caught

Which scaler when

a practical cheat sheet

Bonus quiz

one more check

BACKUP · HOW THE TOKEN WAS CAUGHT

Auditing a model for shortcuts

👁️ Look where it looks

Saliency maps highlight the pixels driving a decision. The pneumonia model lit up **the image corner** — not the lungs. Case closed.

✂️ Remove the suspect

Ablation: crop or mask the token region and re-test. Accuracy collapses? The token was doing the work.

🌍 Test out of habitat

Evaluate on data from a *different* hospital, city, or camera. Shortcuts are local; real signal travels.

The unifying idea: a suspiciously good score is a hypothesis to attack, not a result to celebrate — Class 3's train-vs-test gap thinking, aimed at *what* was learned rather than *how much*. Your data is the model's whole world; audit the world.

BACKUP · WHICH SCALER WHEN

A practical scaling cheat sheet

Min-max (0–1) — bounded, known ranges: pixels, percentages, box coordinates. Fragile if a wild outlier stretches the range.

Z-score — open-ended or outlier-prone columns: distances, prices, counts. The robust default when unsure.

Log transform — heavily skewed positives (income, city sizes): compresses the long tail before scaling.

None at all — tree-based models (Class 12!) split on thresholds and **don't care about scale**. Scaling is for distance- and gradient-based learners.

Meta-rule: the scaler is part of the experiment — fit it on the **training pile only**, then apply to validation/test. Fitting on everything leaks the test set's range into training: data leakage's sneakiest disguise (Class 3's golden rule, again).

BACKUP · BONUS CHECK

Quick check · tap an answer

After cleaning, your model's accuracy DROPPED from 96% to 81%. Your teammate says the cleaning broke something. What's the sharper interpretation?

- Undo the cleaning — 96% was better
- The cleaning removed a shortcut (duplicates, leaks, or a token-like signal): 96% was partly fake, and 81% is the honest baseline
- Models always get worse over time
- Accuracy doesn't matter